Deep Learning - Forward and Back Propagation

Refai Arafat Rossi

October 2026

1 Introduction

[Output Layer Parameters & Training]Output Layer Parameters and Backpropagation Understanding Trainable Weights v_1, v_2, c and Gradient Updates Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

height

1. Clarifying Output-Layer Parameters Core Misconception to Avoid Output-layer weights v_1, v_2 and bias c are **NOT** calculated or derived from input-to-hidden parameters (w_1, w_2, b_1, b_2).

0.5 **Hidden-Layer Parameters:**

- w_1, w_2: Input-to-hidden weights
- b_1, b_2: Hidden neuron biases
- Govern transformation: $x \to h$

0.5 **Output-Layer Parameters:**

- v_1, v_2: Hidden-to-output weights
- c: Output neuron bias
- Govern transformation: $h \to \hat{y}$

Takeaway v_1, v_2, c are independent, learnable model parameters. They are initialized randomly (or with small values) at the start of training and optimized via backpropagation.

2. Problem Setup and Given Initial Values Consider a simplified feedforward network architecture:

- **Input Features:** $x_1 = 3, \quad x_2 = -1$
- **Given Hidden Weights:** $w_1 = 1, \quad w_2 = 2$

- **Given Hidden Biases:** $b_1 = 1, \quad b_2 = 1$
- **Ground Truth Target:** $y = 1.0$

Hidden Layer Function Formulation:

$$h_1 = ReLU(w_1 x_1 + b_1), \qquad h_2 = ReLU(w_2 x_2 + b_2)$$

Architectural Note In a fully connected layer with 2 inputs and 2 hidden units, there are normally 4 input-to-hidden weights ($W \in R^{2\times 2}$). Here we use a 1-to-1 connection mapping for step-by-step clarity.

3. Step 1: Calculate Hidden-Layer Outputs (h_1, h_2) 0.5 Hidden Neuron 1 Computation Pre-activation:

$$z_1 = w_1 x_1 + b_1 = (1)(3) + 1 = \mathbf{4}$$

Apply ReLU activation:

$$h_1 = \max(0, z_1) = \max(0, 4) = \mathbf{4}$$

0.5 Hidden Neuron 2 Computation Pre-activation:

$$z_2 = w_2 x_2 + b_2 = (2)(-1) + 1 = \mathbf{-1}$$

Apply ReLU activation:

$$h_2 = \max(0, z_2) = \max(0, -1) = \mathbf{0}$$

Hidden Representation Output Vector:

$$\mathbf{h} = h_1$$

$\mathrm{h}_2 = \ 4$
0

4. Step 2: Where Output Parameters v_1, v_2, c Enter The output layer combines hidden neuron activations linearly for regression:

$$\hat{y} = v_1 h_1 + v_2 h_2 + c$$

Substituting our calculated hidden outputs ($h_1 = 4, h_2 = 0$):

$$\hat{y} = v_1(4) + v_2(0) + c = \mathbf{4v_1} + \mathbf{c}$$

0.55 **Key Observations:**

- v_1 scales the active feature ($h_1 = 4$).

- v_2 contributes 0 to prediction because $h_2 = 0$ (deactivated by ReLU).
- c shifts the baseline output.

0.45 Why Initialize Early? Forward propagation cannot complete without numerical values for v_1, v_2, c. We pick initial estimates to start the training loop.

5. Step 3: Forward Pass Loss Evaluation Let's set standard illustrative starting values:

$$v_1 = 0.2, \quad v_2 = 0.2, \quad c = 0.0$$

1. Compute Network Prediction ($\hat{y}$):

$$\hat{y} = 4(0.2) + 0(0.2) + 0 = 0.8 + 0 + 0 = \mathbf{0.8}$$

2. Compute Prediction Error (e):

$$e = \hat{y} - y = 0.8 - 1.0 = \mathbf{-0.2}$$

3. Evaluate Half-Squared Error Loss (L):

$$L = \frac{1}{2}(\hat{y} - y)^2 L = \frac{1}{2}(0.8 - 1.0)^2 = \frac{1}{2}(-0.2)^2 = \frac{1}{2}(0.04) = \mathbf{0.02}$$

6. Step 4: Backpropagation Gradients for Output Parameters Using the chain rule, differentiate loss $L = \frac{1}{2}(\hat{y} - y)^2$ with respect to output parameters:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.8 - 1.0 = -0.2$$

0.33 Gradient w.r.t. v_1 $\partial L \overline{\partial v_1 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1} = (\hat{y} - y) h_1 = (-0.2)(4) = \mathbf{-0.8}}$

0.33 Gradient w.r.t. v_2 $\partial L \overline{\partial v_2 = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_2} = (\hat{y} - y) h_2 = (-0.2)(0) = \mathbf{0.0}}$

0.33 Gradient w.r.t. c $\partial L \overline{\partial c = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial c} = (\hat{y} - y)(1) = \mathbf{-0.2}}$

7. Step 5: Gradient Descent Parameter Updates Apply parameter update rule $\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$ with learning rate $\eta = 0.1$:

- **Update v_1:**

$$v_{1,new} = 0.2 - 0.1(-0.8) = 0.2 + 0.08 = \mathbf{0.28}$$

- **Update v_2:**

$$v_{2,new} = 0.2 - 0.1(0.0) = 0.2 - 0.0 = \mathbf{0.20} \quad (\textit{Unchanged!})$$

- **Update c:**

$$c_{new} = 0.0 - 0.1(-0.2) = 0.0 + 0.02 = \mathbf{0.02}$$

Metric	Initial Pass	Updated Pass	Change
Prediction ($\hat{y}$)	0.8000	1.1400	Moved toward target 1.0
Loss (L)	0.0200	0.0098	**Reduced by** 51.0%

Insight on Inactive Neurons v_2 receives zero gradient update because its corresponding hidden neuron $h_2 = 0$. Inactive neurons transmit no error information back to their incoming weights.

8. Step 6: Verifying Updated Output Performance Re-evaluate forward propagation using new parameters ($v_{1,new} = 0.28, v_{2,new} = 0.20, c_{new} = 0.02$):

- Hidden Activations: $h_1 = 4, \quad h_2 = 0$
- New Prediction: $\hat{y}_{new} = (0.28)(4)+(0.20)(0)+0.02 = 1.12+0+0.02 = \mathbf{1.14}$
- Target $y = 1.0 NewErrore_{new} = 1.14 - 1.0 = +0.14$
- New Loss: $L_{new} = \frac{1}{2}(0.14)^2 = \mathbf{0.0098}$

9. Complete Architecture & Data Flow Diagram [x=2.2cm, y=1.3cm, ¿=stealth] [circle, draw, fill=blue!10, minimum size=0.9cm] (x1) at (0, 1) $x_1 = 3$; [circle, draw, fill=blue!10, minimum size=0.9cm] (x2) at (0, -1) $x_2 = -1$;
[circle, draw, fill=green!10, minimum size=0.9cm, align=center] (h1) at (1.5, 1) $h_1 = 4$; [circle, draw, fill=green!10, minimum size=0.9cm, align=center] (h2) at (1.5, -1) $h_2 = 0$;
[circle, draw, fill=red!10, minimum size=0.9cm] (y) at (3, 0) $\hat{y}$;
[-¿, thick] (x1) – node[above] $w_1 = 1, b_1 = 1$ (h1); [-¿, thick] (x2) – node[below] $w_2 = 2, b_2 = 1$ (h2);
[-¿, thick] (h1) – node[above right] $v_1 = 0.28$ (y); [-¿, thick] (h2) – node[below right] $v_2 = 0.20$ (y);
[draw=none] (c) at (3, -1.2) Bias $c = 0.02$; [-¿, dashed] (c) – (y);

Summary Equation

$$\hat{y} = v_1 \cdot ReLU(w_1 x_1 + b_1) + v_2 \cdot ReLU(w_2 x_2 + b_2) + c$$

10. Summary Takeaways for Students

1. **Parameter Separation:** Input-to-hidden parameters (w_i, b_i) determine feature representations (h_i). Output-layer parameters (v_i, c) dictate how those representations combine into final predictions ($\hat{y}$).

2. **Trainable Nature:** Output parameters are independent learnable quantities initialized prior to training; they are not derived mathematically from input weights.

3. **Gradient Masking by Activations:** When a neuron's activation is zero ($h_2 = 0$), its outgoing weight gradient becomes zero ($\frac{\partial L}{\partial v_2} = 0$), temporarily halting updates for that connection.

4. **The Core Learning Loop:**

   $$Inputs\mathbf{x} \longrightarrow Hidden\mathbf{h} \longrightarrow Output\hat{y} \longrightarrow LossL \longrightarrow Gradients\nabla L \longrightarrow Updates\theta_{new}$$